

Availability, SLI, SLO, SLA and Reliability

How distributed systems measure reliability, set targets for it, promise it to customers, and recover when it breaks.

Availability

Nines

Error budget

SLI

SLO

SLA

MTBF

MTTR

The progression we will follow

01	Availability	How often can users actually use the service?
02	The nines	Shorthand for targets like 99.9% or 99.99%
03	Error budget	How much failure the target permits
04	SLI	The measurement of what actually happened
05	SLO	The internal target for that measurement
06	SLA	The formal commitment made to customers
07	MTBF and MTTR	How often we fail, and how fast we recover

Availability: is the system there when users need it?

The percentage of time a system is operational and accessible when users try to use it.

$$\text{Availability} = \text{Uptime} \div \text{Total time} \times 100 = 1 - (\text{Downtime} \div \text{Total time})$$

100 h

Total time the service should run

1 h

Downtime, so 99 h of uptime

99%

Availability = $99 \div 100 \times 100$

A running server is not an available service

The health check says fine

GET /health → 200 OK

The process is alive and answers its probe.

Users see failures

GET /users → 500

GET /orders → 500

GET /payments → 500

Every real operation fails. The service is down.

Measure **user-facing availability**, not machine uptime. Health checks that are too shallow miss real failures; too deep, and one non-critical dependency marks the whole service unhealthy.

Request-based availability for an API

$\text{Availability} = \text{Successful eligible requests} \div \text{Total eligible requests} \times 100$

1,000,000

Total eligible requests

500

Failed, so 999,500 succeeded

99.95%

$999,500 \div 1,000,000$

Why “eligible” requests?

GET /users/invalid-id → 400 Bad Request is the client’s mistake, not the service’s. A production SLI must define precisely which requests count and what success means.

Outages are rarely all-or-nothing

Partial outage

Region A · 95% of users · healthy

Region B · 5% of users · unavailable

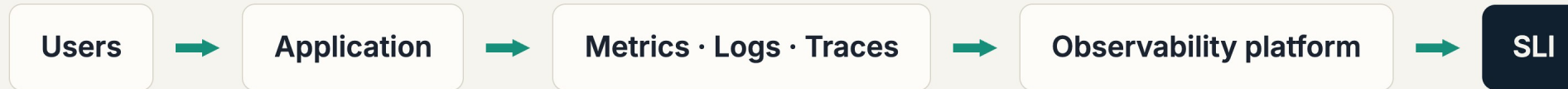
Not globally down, yet 5% of users are in an outage. Slice availability by region, zone, endpoint, customer, request type, location or dependency.

Latency counts too

Response	Verdict
200 in 100 ms	Good
200 in 500 ms	Good
200 in 1.8 s	Good
200 in 5 s	Not good for the SLO
500 in 100 ms	Failure
Timeout	Failure

Definition in use: a request is successful if it returns a valid response within 2 seconds.

How availability is measured in practice



Typical tools

Tool	Primary purpose
Prometheus	Metrics
Grafana	Visualization
Loki / ELK	Logs
Tempo / Jaeger	Traces
Alertmanager	Alerting

From metrics to an SLI

```
http_requests_total
http_requests_failed_total
http_request_duration_seconds
```

```
SLI = rate(2xx) ÷ (rate(2xx)
+ rate(5xx))
```

The exact expression depends on the service's definition of success.

The nines: every extra nine cuts downtime tenfold

Availability	Common name	Approx. downtime per year
99%	Two nines	3.65 days
99.9%	Three nines	8.76 hours
99.99%	Four nines	52.56 minutes
99.999%	Five nines	5.26 minutes
99.9999%	Six nines	31.5 seconds

Assumes a 365-day year. Each row allows roughly one tenth of the downtime of the row above it.

Worked example: what 99.99% actually allows

STEP 1 · UNAVAILABLE SHARE

$100\% - 99.99\%$
 $= 0.01\%$
 $= 0.0001$

STEP 2 · MINUTES IN A YEAR

$365 \times 24 \times 60$
 $= 525,600 \text{ min}$

STEP 3 · ALLOWED DOWNTIME

$525,600 \times 0.0001$
 $= 52.56 \text{ min per year}$

The measurement window matters

The same 99.99% over 30 days allows only $30 \times 24 \times 60 \times 0.0001 = 4.32 \text{ minutes}$. Always ask: **99.99% over what period?**

The error budget is the failure you are allowed

$$\text{Error budget} = 100\% - \text{SLO}$$

For an availability SLO of 99.99%:

0.01%

Permitted unavailability

52.6 min

Time budget per year

100

Failed eligible requests per million
(1,000,000 × 0.0001)

Once a target is chosen, unreliability becomes a quantity you can **spend, track and run out of**.

Spending the budget: 52.6 minutes for the year

Incidents this year

Incident	Minutes
Database incident	20
Bad deployment	10
Network incident	5
Total consumed	35
Remaining (52.6 - 35)	17.6

What the budget buys

A quantitative way to balance risk against progress:

- New features
- Deployments
- Experiments
- Operational risk
- Reliability work

The goal is not zero failures. **Failures are acceptable within a defined budget.**

SLI — Service Level Indicator: what actually happened

A **quantitative measurement** of some aspect of a service's performance. An SLI of 99.97% means 99.97% of eligible requests met the defined success criterion.

Availability

Share of eligible requests that complete successfully

Latency

99% of requests complete within 500 ms

Throughput

10,000 requests per second processed

Freshness

99% of inventory data is under 5 minutes old

Durability

Stored data is preserved with extremely high reliability

The SLI is the number reality produces. The next two concepts are what we want that number to be, and what we promise about it.

Where SLI data comes from

SLIs are calculated from **production evidence**: real users, through the application, into metrics, logs and traces, then aggregated into one number.

Application metrics

Counters and histograms the app exposes: total requests, failed requests, request duration

Load balancer or gateway

Request counts, status codes, response times, connection errors, target health

Logs

POST /payments
status=200
latency=182ms
Aggregated into rates

Distributed traces

Client → Gateway →
Order → Payment →
Database. Best for explaining *why* an SLI degraded

Synthetic monitoring

A scripted journey:
login → search → add to cart → checkout.
Catches problems even when real traffic is low

SLO — Service Level Objective: what we want to achieve

TARGET VS REALITY

SLI = 99.97%

SLO = 99.95%

Actual performance is above the target, so the SLO is currently being met.

A precise SLO, not “our API should be fast”

99% of valid checkout requests complete successfully within 2 seconds over a rolling 30-day window.

- **99%** — the target
- **valid checkout requests** — the population being measured
- **complete successfully** — what counts as good
- **within 2 seconds** — what is measured
- **rolling 30-day window** — the measurement window

An SLO does not have to be 100%

Zero failure is usually impractical and expensive. Teams choose a target such as 99.9%, 99.95% or 99.99% based on:

User needs

Business requirements

Technical feasibility

Cost

Risk

Consequences of failure

The chosen SLO fixes the error budget. For SLO = 99.95%:

0.05%

Error budget = $100\% - 99.95\%$

500

Unsuccessful eligible requests per million
($1,000,000 \times 0.0005$)

21.6 min

Downtime per 30 days if the SLO is time-based
($43,200 \times 0.0005$)

SLA — Service Level Agreement: what we promise the customer

A **formal agreement** between a service provider and a customer about the service level, often with contractual consequences if the commitment is not met.

INTERNAL SLO

99.95%

What engineering aims for

The gap is deliberate
margin between the
engineering target and
the external
commitment

CUSTOMER SLA

99.90%

What the contract guarantees

SLO missed → an engineering and reliability concern

SLA missed → a contractual and business consequence:
service credits, refunds, other remedies defined in
the contract

SLI, SLO, SLA and error budget side by side

Concept	Question it answers	Example
SLI	What happened?	99.97% successful
SLO	What do we target?	$\geq 99.95\%$
SLA	What do we formally promise customers?	$\geq 99.90\%$
Error budget	How much failure does the SLO allow?	0.05%

SLI measures reality.

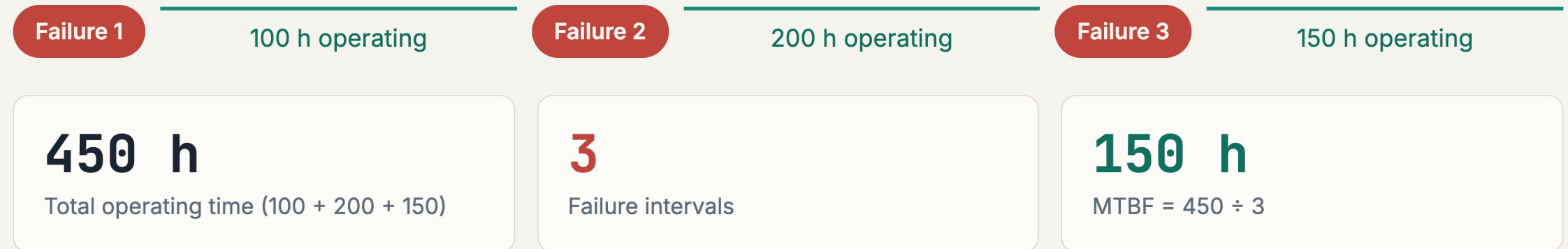
SLA defines the external commitment.

SLO sets the target.

Error budget defines the allowed failure under the SLO.

MTBF — Mean Time Between Failures: how often do we fail?

$$\text{MTBF} = \text{Total operating time} \div \text{Number of failures}$$



MTTR — Mean Time To Repair: how fast do we recover?

$\text{MTTR} = \text{Total recovery time} \div \text{Number of failures}$

Failure 1

30 min

Failure 2

10 min

Failure 3

20 min



$(30 + 10 + 20) \div 3$

20 min

Repair, recover or restore?

Organizations use "MTTR" differently. Always check, and state, the definition being used before comparing numbers.

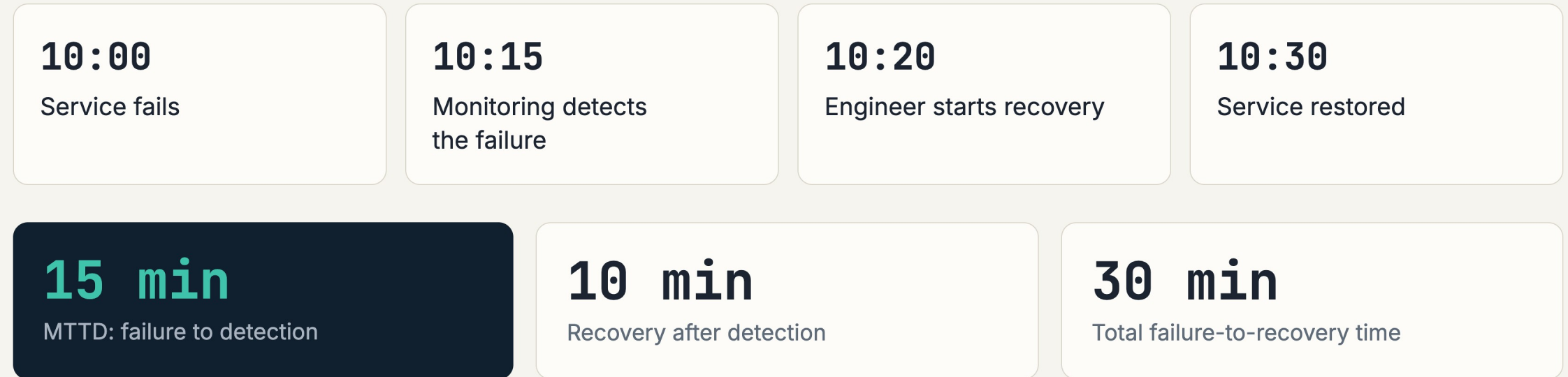
Two levers on availability: fail less, recover faster

$$\text{Availability} \approx \text{MTBF} \div (\text{MTBF} + \text{MTTR})$$

Scenario	MTBF	MTTR	Availability
Baseline	1,000 h	1 h	$1000 \div 1001 \approx 99.90\%$
Recover faster	1,000 h	10 min (0.1667 h)	$\approx 99.983\%$
Fail less	10,000 h	1 h	$10000 \div 10001 \approx 99.99\%$

Faster recovery raises availability significantly **even when failures still happen**. Making failures rarer does the same. Both levers are available to every team.

MTTD — Mean Time To Detect: how long until we notice?



Failure → MTTD → detection → recovery process → MTTR → service restored. Because MTTR is defined differently across organizations, every metric needs an explicit definition.

Improving MTBF and MTTR

Fail less — raise MTBF

- **Redundancy** — no single points of failure
- **Better testing** — catch bugs before production
- **Safer deployments** — canary, blue/green, gradual rollouts
- **Capacity planning** — avoid CPU, memory, disk and connection-pool exhaustion
- **Fault isolation** — stop one component from cascading into others

Recover faster — lower MTTR

- **Monitoring** — detect failures quickly
- **Good alerting** — the right people, only actionable failures
- **Automated rollback** — error rate rises, previous version returns
- **Automated failover** — a replica becomes active on primary failure
- **Runbooks** — documented recovery for known incidents
- **Automation** — no engineer repeating recovery steps by hand

The complete mental model

Fail less → MTBF

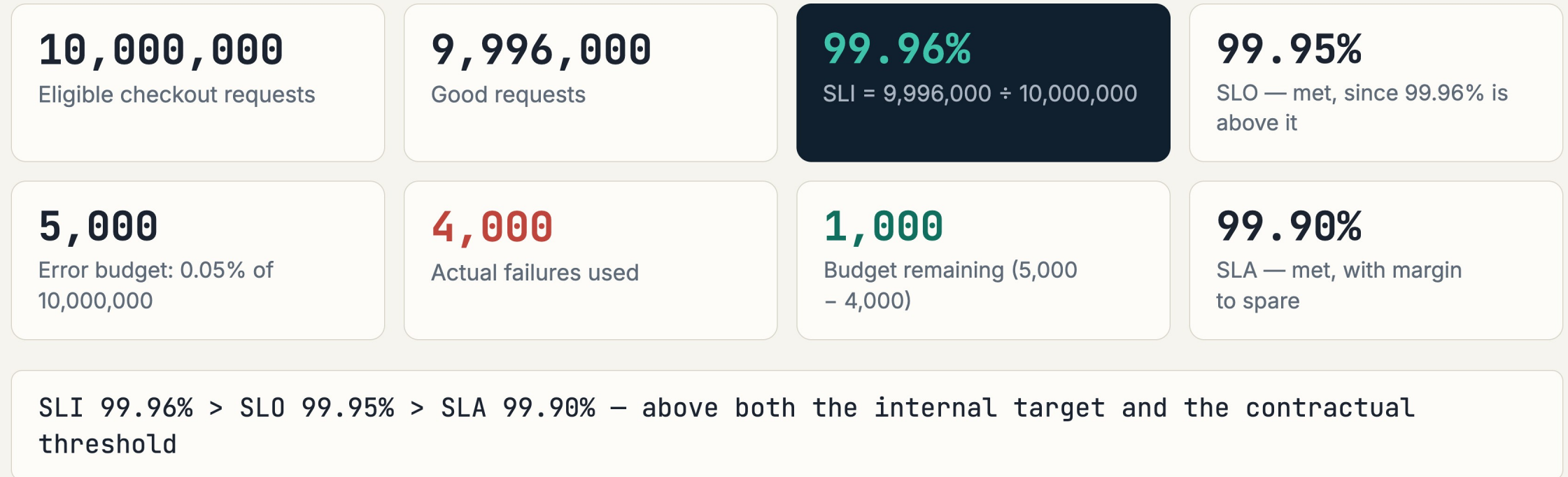
Recover faster → MTTR



Availability	How often can users successfully use the service?
SLI	What actually happened?
SLO	What do we want?
Error budget	How much failure can we tolerate?
SLA	What do we formally promise customers?

One complete example: an e-commerce checkout service

A checkout request is successful when it is **valid**, returns a **successful response**, and completes **within 2 seconds**. Over a 30-day period:



The core formula set

Availability

$\text{Uptime} \div \text{Total time} \times 100$

Request-based availability

$\text{Good requests} \div \text{Eligible requests} \times 100$

Error budget

$1 - \text{SL0}$

Downtime budget

$\text{Total time} \times (1 - \text{SL0})$

MTBF

$\text{Operating time} \div \text{Number of failures}$

MTTR

$\text{Total recovery time} \div \text{Number of failures}$

Availability from failure mechanics

$\text{Availability} \approx \text{MTBF} \div (\text{MTBF} + \text{MTTR})$

Key takeaways

Availability

How often users can successfully use the service

Nines

Shorthand for targets such as 99.9%, 99.99% and 99.999%

Error budget

How much failure the target permits

SLI

The actual quantitative measurement of service performance

SLO

The internal target for that measurement

SLA

The formal external commitment to customers

MTBF

How long the system operates between failures

MTTR

How long it takes to recover from a failure

MTTD

How long it takes to detect a failure

Interview quick answers

If asked...	Answer in one breath
What is availability?	How often a service can successfully provide its intended functionality to users.
What are three nines?	99.9% availability — roughly 8.76 hours of downtime per year on a 365-day year.
SLI vs SLO vs SLA?	SLI is the measurement; SLO is the internal target for it; SLA is the formal customer commitment with contractual consequences.
What is an error budget?	The unreliability an SLO permits — for a 99.95% SLO, 0.05% of requests or time.
MTBF vs MTTR?	MTBF measures how frequently failures occur; MTTR measures how quickly the system is restored afterwards.
How do you improve availability?	Increase MTBF by failing less, and decrease MTTR by detecting and recovering faster.

IN ONE SENTENCE

Fail less, recover faster — then measure it, target it, and promise only what you can keep.

Measure it — SLI

Target it — SLO and error budget

Promise it — SLA

Improve it — MTBF and MTTR